

STAT:5400 Section 2.4 — LLM APIs and Prompting

Boxiang Wang

2026-09-16

1 What a Language Model Is

1.1 The one-sentence version

- A **large language model** writes text one token at a time, each token chosen using everything written before it. That has not changed.
- **Pretraining** is exactly that task: predict the next token, over a very large amount of text.
- **Reasoning models** add reinforcement learning after pretraining, rewarded for correct answers rather than for matching text. They think in hidden tokens before replying, which you pay for but never see.
- Modern models are built on the **Transformer** architecture, introduced in 2017.
- A broader sense of the term includes models that also handle images, audio, and video.

Source: Vaswani et al., *Attention Is All You Need*, 2017. <https://arxiv.org/abs/1706.03762>

1.2 History and evolution

- **Early neural sequence models.** ELIZA (1966); recurrent neural networks (1990s).
- **1997 to 2010, better memory.** LSTM (1997) handled longer dependencies in text. Toolkits such as Stanford CoreNLP standardized NLP pipelines.
- **2011 to 2017, deep learning and the Transformer.** Word2Vec (2013) gave efficient word embeddings. The Transformer (2017) became the core architecture of nearly every modern model.
- **2018 onward, foundation models and chatbots.** BERT (2018); the GPT series (2018 to 2024); ChatGPT (2022) brought prompt-based use to everyone. Hugging Face and LLaMA made models easier to share and run.

Source: GeeksforGeeks, *History and Evolution of LLMs*. <https://www.geeksforgeeks.org/blogs/history-and-evolution-of-llms/>

1.3 Two ways a model reaches you

- **Closed weights.** You call an API and never see the weights. [OpenAI GPT](#), [Google Gemini](#), [Anthropic Claude](#).
- **Open weights.** You can download, run, and fine-tune the model yourself, subject to its licence. [Meta Llama](#), [Mistral](#), [Qwen](#).
- The code in this course is written so that the two are interchangeable: the same call works against a hosted model, against the course model, or against a model running on your own machine.

2 Keys, Tokens, and Cost

2.1 API keys

[Section 1.1: Set up your accounts.](#)

- Read the prices: <https://platform.openai.com/docs/pricing>.
- Set a hard budget limit on the account. Do it today, not after the first surprise.

2.2 What a token costs

- Billing is per **token**, counted on both what you send and what comes back.
- One token is roughly 0.75 words. One million tokens is about 3000 double-spaced pages.

This block runs here, because arithmetic needs no network.

```
prompt <- "Explain what a p-value means in one sentence for a beginner."

words  <- length(strsplit(prompt, "\\s+")[1])
tokens <- round(words / 0.75)
c(words = words, approx_tokens = tokens)
```

 **Edit** — put in a price per million tokens from the pricing page and a number of calls you might make in a homework, and see what the assignment would cost.

```
price_per_million <- 0.15      # dollars, input tokens; check the current page
calls              <- 500

round(tokens * calls / 1e6 * price_per_million, 4)
```

A size you can picture. *The Elements of Statistical Learning* runs to 745 pages.

```
book_tokens <- 745 * 500 / 0.75  # pages, words per page, words per token
book_tokens
```

Half a million tokens. Now send the whole book to a model, once, and read the bill at both ends of the price list.

```
# dollars per million input tokens, September 2026; these change often
per_million <- c(nano = 0.05, pro = 30.00)

round(book_tokens / 1e6 * per_million, 2)
```

- Two cents on the cheapest model, about fifteen dollars on the most expensive. The same words.
- Output costs more than input, usually eight to ten times more, so a model that answers at length is the expensive one.
- A conversation resends its whole history every turn, so you pay for the book again on the next question.

The rule from Section 1.1 still holds. A key is a password. It lives in the environment, never in a file you can see. Read it with `Sys.getenv("OPENAI_API_KEY")` in R or `os.environ` in Python, and never print it.

3 Your First Call

3.1 Hello world

Run in Codespace — these need a key and a network, so they run in the Codespace.

3.1.1 R

```
library(ellmer)

chat <- chat_openai(model = "gpt-5-nano")
chat$chat("Write a one-sentence bedtime story about a unicorn.")
```

3.1.2 Python

```
from dotenv import load_dotenv
from openai import OpenAI
load_dotenv()

client = OpenAI()
response = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "user", "content":
        "Write a one-sentence bedtime story about a unicorn."}],
)
print(response.output_text)
```

Run it twice. The two answers differ, because the model samples. We come back to why in Section 3.1.

3.2 The model has no memory

- Every request is **stateless**. The model sees only what you send in that request.
- Ask about “the university I just mentioned” in a second, separate call and it has no idea.

3.2.1 R

Two `chat` objects are two separate conversations. One `chat` object is a memory.

```
library(ellmer)

first <- chat_anthropic()           # reads ANTHROPIC_API_KEY
first$chat("Write a paragraph about the Department of Statistics and
          Actuarial Science at the University of Iowa.")

second <- chat_anthropic()          # a brand new conversation
second$chat("What is the name of the university I just mentioned?")
```

```
same <- chat_anthropic()
same$chat("Write two sentences about the University of Iowa.")
same$chat("What is the name of the university I just mentioned?")
```

3.2.2 Python

```
r1 = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "user", "content":
        "Write a paragraph about the Department of Statistics and Actuarial "
        "Science at the University of Iowa."}])
print(r1.output_text)

r2 = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "user", "content":
        "What's the name of the university that I just mentioned?"}])
print(r2.output_text)      # it does not know
```

- A conversation is something *you* build, by sending the history back every time.
- `ellmer` does this for you: a `chat` object keeps the turns and resends them.
- Two providers, same behaviour. The statelessness is not a quirk of one company.

3.3 The same code, an open-weight model

Most providers speak the OpenAI format, so only the address and the model name change. That includes open-weight models you run yourself. Your Codespace already has two small ones through Ollama, and they need no key and cost nothing.

3.3.1 R

```
local <- chat_ollama(model = "llama3.2:1b")    # open weights, on this machine
local$chat("Give me 3 distributions in the exponential family.")

course <- chat_openai_compatible(              # the course model on IDAS
    base_url = Sys.getenv("COURSE_MODEL_URL"),
    model    = Sys.getenv("COURSE_MODEL"),
    api_key  = Sys.getenv("COURSE_MODEL_KEY"))
course$chat("Give me 3 distributions in the exponential family.")
```

3.3.2 Python

```
local = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
resp = local.chat.completions.create(
    model="llama3.2:1b",
    messages=[{"role": "user",
                "content": "Give me 3 distributions in the exponential family."}])
print(resp.choices[0].message.content)
```

Compare the three answers. A one-billion-parameter model on your own machine is visibly worse than a hosted one, and that gap is the thing you are paying for.

The small local model is also the fastest way to debug your own code. If your script works against it, the problem was never the network or the key.

4 Prompting

4.1 Rules of thumb

Prompt engineering means writing instructions clear enough that the model does what you need, reliably. The output is random, so this is part art and part science. Nine rules carry most of the weight.

1. Use the latest model.
2. Put the instruction at the beginning, and separate instruction from context with **###** or **"""**.
3. Be specific about context, outcome, length, format, and style.
4. Show the desired output format with an example.
5. Start zero-shot, then few-shot, then fine-tune. In that order.
6. Cut vague adjectives.
7. Say what to do instead of what not to do.
8. For code, use leading words to nudge the model toward a pattern.
9. Ask the model to write the prompt for you, then edit it.

Source: OpenAI prompt guide, <https://platform.openai.com/docs/guides/prompt-engineering>;
Liu et al. (2021), <https://arxiv.org/pdf/2107.13586>

4.2 Roles

A message carries a role, and the roles form a chain of command. The highest level is not yours.

- **platform**: the provider's own rules. You cannot override them, which is why a prompt telling the model to ignore its safety rules does not work.
- **developer**: your instructions. Sent as the `instructions` argument, or as a message with role `developer`.
- **user**: the end user's request, which ranks below your instructions.
- **assistant**: what the model said on earlier turns.

i system or developer?

They are the same level, under two names. `system` is the old name. Since the reasoning models arrived, the role is called `developer`, and current models expect that name. Older code using `system` still works, and most providers quietly translate it, but write `developer` in anything new. In `ellmer` the argument is still called `system_prompt`, and it is sent at this level.

Source: OpenAI model spec, https://model-spec.openai.com/2025-02-12.html#chain_of_command

💡 Predict — the same question, two system prompts. Which one is more useful to a student who has to take an exam?

4.2.1 R

```
chat_a <- chat_openai(model = "gpt-5-nano",
  system_prompt = "You are a concise tutor.")
chat_a$chat("X follows a Uniform(0, 1). What is its variance?")

chat_b <- chat_openai(model = "gpt-5-nano",
  system_prompt = paste("You are a responsible tutor. Help your student",
    "but don't tell them the exact answers.))")
chat_b$chat("X follows a Uniform(0, 1). What is its variance?")
```

4.2.2 Python

```

for system in ["you are a concise tutor.",
               "you are a responsible tutor. Help your student "
               "but don't tell them the exact answers."]:
    response = client.responses.create(
        model="gpt-5-nano",
        input=[{"role": "developer", "content": system},
               {"role": "user", "content":
                "X follows a uniform (0, 1). What is its variance?"}]])
    print(response.output_text, "\n---")

```

4.3 Conversation state

Three ways to give a stateless model a memory.

1. **Pass the previous messages yourself**, as a list with roles.
2. **Append each reply to a history list** and send the whole list next time.
3. **Let the provider store it**, with `previous_response_id` in Python or a `chat` object in `ellmer`.

```

response = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "user",      "content": "knock knock."},
           {"role": "assistant", "content": "Who's there?"},
           {"role": "user",      "content": "Felix"}])
print(response.output_text)

```

i The context window

Everything you resend is paid for again, and there is a ceiling.

The **context window** is the maximum number of tokens in one request, counting what you send, what comes back, and for some models the hidden reasoning. A long conversation eventually hits it, and the output gets truncated. Model limits are listed at <https://platform.openai.com/docs/models>.

4.4 Few-shot learning

- **Zero-shot**: you describe the task. **Few-shot**: you also show a handful of worked examples.
- The model reads the pattern from the examples and applies it. No fitting, no gradient, nothing stored.

- Diverse examples generalize better than similar ones.

Source: Brown et al. (2020), *Language Models are Few-Shot Learners*. <https://arxiv.org/pdf/2005.14165>

A classifier for statistical claims, built entirely out of three examples:

```
INSTRUCTIONS = """# Identity

You are a helpful assistant that labels short statistical claims as
Supported, Unsupported, or Unclear based on the evidence provided.

# Instructions

* Only output a single word, with no additional formatting or commentary.
* Your response must be one of "Supported", "Unsupported", or "Unclear".

# Examples

<stat_claim id="example-1">
The new drug lowered systolic blood pressure
(mean difference -5.2 mmHg, 95% CI -8.1 to -2.3, p=0.001).
</stat_claim>
<assistant_response id="example-1">Supported</assistant_response>

<stat_claim id="example-2">
This study proves the two groups are different
(mean difference 1.1, 95% CI -0.6 to 2.8, p=0.20).
</stat_claim>
<assistant_response id="example-2">Unsupported</assistant_response>

<stat_claim id="example-3">
The treatment improved outcomes, but only a trend was observed (p=0.07).
</stat_claim>
<assistant_response id="example-3">Unclear</assistant_response>
"""

ALLOWED = {"Supported", "Unsupported", "Unclear"}

def label_claim(claim):
    resp = client.responses.create(model="gpt-5-nano",
                                  instructions=INSTRUCTIONS, input=claim)
    label = (resp.output_text or "").strip().split()[0].strip('"\'.,;:')
```

```

if label not in ALLOWED:
    raise ValueError(f"Unexpected label {label!r}")
return label

for c in ["We can conclude there is no effect because p=0.40.",
         "The results show a significant increase (95% CI 0.1 to 0.4).",
         "The intervention might help, but the CI crosses zero (95% CI -0.2 to 0.3)."]:
    print(label_claim(c))

```

The function does not trust the reply. It checks that the word is one of the three allowed labels, and raises an error if it is not. A model can return anything, including a polite paragraph where you asked for one word.

4.5 Sampling controls

The model produces a probability for every token in its vocabulary, then draws one.

- **Temperature** controls how random that draw is. At 0 the model always takes the highest-probability token, which is called greedy decoding. As it rises, the distribution flattens.
- **Top-p**, or nucleus sampling, keeps the smallest set of tokens whose probabilities add to p, then draws from those.
- Rules of thumb: temperature 0.2 with top-p 0.95 for coherent output; 0.9 and 0.99 for creative output; 0 when the task has one right answer.
- Both extremes can produce a repetition loop. Too low and the model returns to the same text; too high and a random word leads back to a previous state.

i A version trap worth knowing

Some models refuse these settings. Requests to the GPT-5 family, including `gpt-5-nano`, raise an error if `temperature` or `top_p` is included. To set them you need a model that supports them, with reasoning effort turned off.

5 Structured Output

5.1 Ask for a type, not a string

- A model returns text. Text is the wrong thing to hand to `lm` or to a plot.
- Both interfaces let you state a **schema** up front, and return an object that matches it.
- This replaces the check-the-string code in the few-shot example above.

5.1.1 R

```
review <- type_object(  
  label      = type_enum("One of Supported, Unsupported, Unclear.",  
                        c("Supported", "Unsupported", "Unclear")),  
  reason     = type_string("One short sentence."),  
  p_value    = type_number("The p-value mentioned, or -1 if none.))  
  
chat <- chat_openai(model = "gpt-5-nano")  
chat$chat_structured(  
  "This study proves the two groups differ (p = 0.20).",  
  type = review)
```

5.1.2 Python

```
from pydantic import BaseModel  
from typing import Literal  
  
class Review(BaseModel):  
    label: Literal["Supported", "Unsupported", "Unclear"]  
    reason: str  
    p_value: float  
  
resp = client.responses.parse(  
    model="gpt-5-nano",  
    input="This study proves the two groups differ (p = 0.20).",  
    text_format=Review)  
print(resp.output_parsed.label, resp.output_parsed.p_value)
```

6 Tool Calling

6.1 Giving the model a function to call

- A model cannot compute. It can only write text that looks like a computation.
- A **tool** is one of your functions, described well enough that the model can decide when to call it and with what arguments.
- You run the function. The model never touches your data or your session.

The loop has four steps, and you will meet it again in Section 7.

1. You send the question and the list of tools.
2. The model replies with a request: this tool, these arguments.
3. Your code runs the function and sends the result back.
4. The model uses the result to answer, or asks for another tool.

6.2 Watch a small model fail, then fix it

In 2024 chatbots were widely reported to answer that 9.11 is larger than 9.9. Small models still do it. They are reading the digits the way a version number is read, not the way a number is read.

Run in Codespace — needs a model. Use the open-weight one already in your Codespace.

6.2.1 R

```
library(ellmer)

# no tool: the model answers from text alone
bare <- chat_ollama(model = "llama3.2:1b")
bare$chat("Which is larger, 9.9 or 9.11? Answer with just the number.")

# one tool: now the comparison happens in R
larger <- function(a, b) if (a > b) a else if (b > a) b else "equal"

armed <- chat_ollama(model = "llama3.2:1b",
  system_prompt = "Use the tool to compare numbers. Never compare them yourself.")
armed$register_tools(list(
  tool(larger, "Return the larger of two numbers.",
    arguments = list(a = type_number("First number."),
      b = type_number("Second number."))))))
armed$on_tool_request(function(req) cat("-> ", req$name, deparse1(req@arguments), "\n"))
armed$chat("Which is larger, 9.9 or 9.11?")
```

6.2.2 Python

```
import json
local = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

TOOLS = [{"type": "function", "function": {
  "name": "larger",
```

```

    "description": "Return the larger of two numbers.",
    "parameters": {"type": "object",
                    "properties": {"a": {"type": "number"}, "b": {"type": "number"}},
                    "required": ["a", "b"]}}}

msgs = [{"role": "system", "content": "Use the tool to compare numbers."},
        {"role": "user", "content": "Which is larger, 9.9 or 9.11?"}]

r = local.chat.completions.create(model="llama3.2:1b", messages=msgs, tools=TOOLS)
for c in r.choices[0].message.tool_calls or []:
    args = json.loads(c.function.arguments)
    print("->", c.function.name, args, "=>", max(args["a"], args["b"]))

```

i What the tool actually fixed

Nothing about the model changed. The work moved.

The comparison now happens in R or in Python, where $9.9 > 9.11$ is arithmetic. The model's only job is to notice that a comparison is needed and to hand over two numbers. That is the whole argument for tools. Anything a model can get wrong by writing text, give to code instead.

⚠ If the small model ignores the tool

One-billion-parameter models are unreliable at tool calling. If yours answers without calling **larger**, that is the model, not your code. Point the same script at the course model or at **gpt-5-nano** and it will call the tool every time. Use the small one to see the mechanics, and a larger one when you need it to work.

6.3 A two-tool example

The question is “what is the average weight of these cars?”. The model does not know the column names, so it has to look before it can compute.

Run in Codespace — needs a model.

6.3.1 R

```

library(ellmer)
Cars <- read.delim("notes/section2/data/Cars.dat")

```

```

column_names <- function() paste(names(Cars), collapse = ", ")
column_mean  <- function(column) mean(Cars[[column]])

chat <- chat_openai(model = "gpt-5-nano",
  system_prompt = "Use the tools to compute. Never guess a number.")

chat$register_tools(list(
  tool(column_names,
    "List the column names of the cars data set. Call this first if unsure.",
    arguments = list()),
  tool(column_mean,
    "Return the mean of one numeric column of the cars data set.",
    arguments = list(column = type_string("Exact column name."))))))

chat$on_tool_request(function(req)
  cat("-> ", req$name, deparse1(req$arguments), "\n"))

chat$chat("What is the average weight of these cars?")

```

6.3.2 Python

```

import json, pandas as pd
Cars = pd.read_csv("notes/section2/data/Cars.dat", sep="\t")

TOOLS = [
  {"type": "function", "name": "column_names",
   "description": "List the column names of the cars data set.",
   "parameters": {"type": "object", "properties": {},
                  "required": [], "additionalProperties": False},
   "strict": True},
  {"type": "function", "name": "column_mean",
   "description": "Return the mean of one numeric column of the cars data set.",
   "parameters": {"type": "object",
                  "properties": {"column": {"type": "string"}},
                  "required": ["column"], "additionalProperties": False},
   "strict": True},
]

def call_tool(name, args):
  if name == "column_names": return ", ".join(Cars.columns)
  if name == "column_mean":  return float(Cars[args["column"]].mean())

```

```

messages = [{"role": "user", "content": "What is the average weight of these cars?"}]
for step in range(5):
    resp = client.responses.create(model="gpt-5-nano", input=messages,
                                   tools=TOOLS, parallel_tool_calls=False)

    messages += resp.output
    calls = [o for o in resp.output if o.type == "function_call"]
    if not calls:
        print(resp.output_text); break
    for c in calls:
        result = call_tool(c.name, json.loads(c.arguments or "{}"))
        print("->", c.name, c.arguments, "=>", result)
        messages.append({"type": "function_call_output",
                          "call_id": c.call_id, "output": json.dumps(result)})

```

👉 By hand — the correct answer is the mean of the **Weight** column, which you can compute yourself in one line. Run the agent and check that the number it reports is that number. Done when you can say which tools it called, in which order, and whether it ever guessed.

7 Working with a Coding Agent

7.1 Git

Command	What it tells you
<code>git status</code>	which files changed
<code>git diff</code>	exactly which lines changed
<code>git add .</code> and <code>git commit -m "..."</code>	save a checkpoint
<code>git log --oneline</code>	the checkpoints so far
<code>git restore <file></code>	throw away everything since the last checkpoint

- Commit before you let an agent touch anything. That commit is your undo button.
- `git diff` is how you see what it did, line by line. `/review` reads the same thing.
- A `.gitignore` lists what never enters the repository: `.env`, large data, rendered output. The `.env` line is not optional. It is what stops a key reaching GitHub.

7.2 Installing Codex and Claude Code

Both are command-line programs. One line installs either one and puts the command on your PATH, so you can type it from any folder.

```
# Claude Code
curl -fsSL https://claude.ai/install.sh | bash

# Codex
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

- If you already use Node, `npm install -g @anthropic-ai/claude-code` and `npm install -g @openai/codex` do the same job. The installers above carry their own runtime, so Node is not required.
- Check it worked with `claude --version` or `codex --version`. If the shell says command not found, open a new terminal. If it still fails, the install folder is not on your PATH, and you add it in `~/.zshrc` or `~/.bashrc`.
- The first run asks you to sign in. Claude Code takes a Claude subscription or `ANTHROPIC_API_KEY`. Codex takes a ChatGPT account or `OPENAI_API_KEY`.
- Both also install inside the Codespace terminal, so you can try them without touching your own machine.

7.3 Let the agent set it up

Set up git before you start an agent, every time. Make it a habit.

```
cd ~/projects/my-analysis    # the folder you want to work in
claude                       # or: codex
```

 Vibe — once it starts, ask for something like this.

Initialize a git repository here. Write a `.gitignore` for a project that uses both R and Python, excluding `.env`, `.Rhistory`, `.Rproj.user`, `__pycache__`, and rendered output. Then make the first commit.

Now check it yourself, before you believe any of it.

```
git log --oneline           # exactly one commit?
git status                  # working tree clean?
cat .gitignore              # is .env actually in there?
git ls-files | grep .env    # nothing should come back
```


That last line is the one that matters. If a `.env` file was committed, the agent has just put a key into your history, and deleting the file later does not remove it. Done when you can say what it committed, what it left out, and how you know.

7.4 Chat, API, and coding agents

Three ways to reach the same model, and they are not interchangeable.

- **Chat.** You type into a web page. Fast, and nothing about it is reproducible.
- **API.** You write the call inside a script that can be rerun, versioned, and read by someone else. That is this page.
- **Coding agent.** The model runs the loop on your machine. It reads your files, writes new ones, runs commands, reads the errors, and tries again. [OpenAI Codex](#) and [Anthropic Claude Code](#) are the two common ones. [Copilot](#) in the Codespace is a lighter version of the same idea.
- The difference that matters: with the API you own the loop, and with a coding agent the loop owns your files.
- All three keep a transcript. That transcript is the only reason anyone can check the work afterwards, and reading one is what Section 7 is about.
- In this course: coding agents are welcome on 🧑🏻 Vibe blocks, not on 🙋 By hand blocks, and whatever they write, you run and check yourself.

You drive both from a terminal. You type ordinary English most of the time, and **slash commands** when you want to steer the tool itself. Type `/` alone to see the list for your version, because it changes often. Five are worth knowing on day one.

Command	What it does	Reach for it when
<code>/help</code>	lists every command	you forget the rest of this table
<code>/init</code>	writes a memory file, <code>CLAUDE.md</code> or <code>AGENTS.md</code>	you start work in a new repository
<code>/model</code>	switches models	a task needs more care, or more speed
<code>/clear</code>	throws the conversation away and starts fresh	the agent is going in circles
<code>/effort</code>	sets how long the model thinks before answering	the task is hard, or the bill is too high
<code>/review</code>	reviews the changes instead of making more	you are about to commit

`/effort` takes `low` through `max`. Low is fast and cheap, high thinks longer. It is a Claude Code command; in Codex the same dial sits inside `/model`. Pairing a small model on low effort for mechanical edits with a large one on high effort for design is the single easiest way to cut what these tools cost.

A session looks like this.

```
> /init
wrote AGENTS.md

> read notes/section2/data/Cars.dat and write an R function
  that returns the mean of any numeric column

... writes R/cars.R, runs it, hits one error, fixes it ...

> /review

> /clear
```

`/init` is the one that matters for this course. The memory file is plain text, and it lives in the repository. The instructions the agent followed end up under version control, next to the code it wrote. That is the whole difference between an AI-assisted analysis someone can reproduce and one they cannot.

Going further: carrying work across sessions

A real task outlives one sitting. Three commands and one file handle it.

- `/compact` replaces a long conversation with a summary and keeps going. Reach for it when the session slows down or the context fills up.
- `/resume` reopens an earlier session. At the shell, `--continue` reopens the last one.
- `/export` writes the whole transcript to a text file.

The file is the part that lasts. Session history sits on one machine and does not follow you to another, so anything worth keeping belongs in the repository: the memory file for the instructions, an exported transcript for the record of what actually happened.

This is the same argument as `set.seed`. A result nobody can get back to is not a result.

7.5 What comes next

- Section 7 turns this loop into an agent: several tools, several steps, and a receipt of everything it did.

- The question that matters for a statistician is not whether the model can call a tool. It is whether you can check what it did afterwards.